



DOCUMENTACIÓN TÉCNICA DE SISTEMA

Proyecto: Ticket Museum v1.0

Área: Ingeniería de Software / Base de Datos

Índice

INTRODUCCIÓN Y GENERALIDADES

- 1.1. Descripción del Sistema.....3
- 1.2. Stack Tecnológico.....3
- 1.3. Requerimientos de Software.....3
- 1.4. Requerimientos de Hardware.....3

MODELADO DE DATOS

- 2.1. Estructura de la Base de Datos (SQL).....4
- 2.2. Conexión a la Base de Datos.....4
- 2.3. Diccionario de Datos.....5
 - 2.3.1. Tabla: TB_VISITANTES
 - 2.3.2. Tabla: TB_VENTAS
- 2.4. Diagrama Entidad-Relación.....6

ARQUITECTURA DEL SOFTWARE

- 3.1. Organización de los Componentes Técnicos (WPF).....7
- 3.2. Métodos Principales del Sistema (Lógica Code-Behind)...8

OPERACIÓN Y MANTENIMIENTO

- 4.1. Requerimientos de Instalación y Despliegue.....10
- 4.2. Manejo de Excepciones y Errores.....10
- 4.3. Mantenimiento Futuro y Escalabilidad.....11

Descripción del Sistema

El sistema Ticket Museum es una aplicación de escritorio desarrollada bajo el framework .NET, diseñada para la gestión de transacciones y flujo de visitantes. Utiliza una arquitectura de capas para separar la lógica de negocio de la interfaz de usuario.

➤ Stack Tecnológico

Para la construcción y mantenimiento de este software se utilizó:

- **Lenguaje:** C# (C-Sharp).
- **Entorno de Desarrollo (IDE):** Visual Studio 2022 / VS Code.
- **Gestor de Base de Datos:** SQL Server

➤ Requerimientos de Software

Para que el sistema funcione, la computadora debe tener:

- **Sistema Operativo:** Windows 10 o Windows 11.
- **Base de Datos:** SQL Server.
- **Librerías:** .NET SDK (versión 8.0 o superior recomendada)

➤ Requerimientos de Hardware

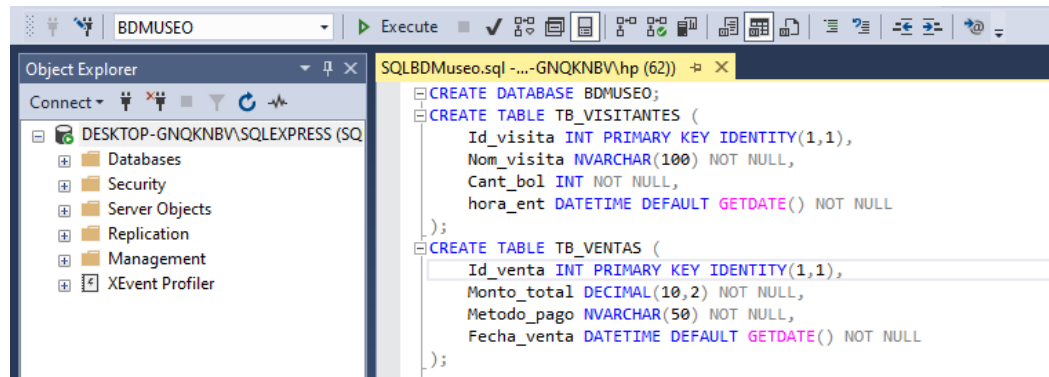
Se recomienda el siguiente equipo para evitar que el sistema se trabe:

- **Memoria RAM:** 4 GB como mínimo (8 GB recomendado).
- **Procesador:** Intel Core i3 o similar.
- **Espacio en Disco:** Al menos 200 MB para la instalación y los datos.

MODELADO DE DATOS

Estructura de la Base de Datos (SQL)

El sistema utiliza una base de datos relacional llamada BDMUSEO. El diseño se realizó directamente en SQL Server, asegurando que la información sea persistente y no se pierda al cerrar el programa.



CONEXIÓN A LA BASE DE DATOS

El sistema está diseñado para conectarse de forma directa a la instancia del servidor local de bases de datos (DESKTOP-GNQKNBV\SQLEXPRESS) para interactuar con las tablas de la taquilla (BDMUSEO). La arquitectura de WPF permite que los módulos operativos (como VentasPage y BitacoraPage) invoquen los componentes de datos mediante controladores asíncronos en el código subyacente (*Code-Behind*), garantizando que la interfaz gráfica no se congele durante las peticiones de lectura y escritura.

Diccionario de datos

Tabla: TB_VISITANTES

Función: Registro de grupos y cantidad de boletos emitidos por visita.

Campo	Tipo de Dato SQL	Restricción / Propiedad	Descripción Técnica
Id_visita	INT	PRIMARY KEY IDENTITY	Identificador único y autoincremental de la visita.
Nom_visita	NVARCHAR(100)	NOT NULL	Nombre del responsable o grupo. Soporta caracteres Unicode.
Cant_bol	INT	NOT NULL	Cantidad total de boletos adquiridos en esta visita.
hora_ent	DATETIME	DEFAULT GETDATE()	Fecha y hora exacta de entrada, asignada por el servidor.

Tabla: TB_VENTAS

Función: Control financiero de las transacciones de boletos.

Campo	Tipo de Dato SQL	Restricción / Propiedad	Descripción Técnica
Id_venta	INT	PRIMARY KEY IDENTITY	Identificador único y autoincremental de la transacción.
Monto_total	DECIMAL(10,2)	NOT NULL	Valor total de la venta, soporta hasta 10 dígitos y 2 decimales.
Metodo_pago	NVARCHAR(50)	NOT NULL	Forma de pago (ej: Efectivo, Tarjeta).
Fecha_venta	DATETIME	DEFAULT GETDATE()	Timestamp automático de la transacción.

DIAGRAMA ENTIDAD-RELACION

Para representar visualmente cómo se almacena la información, se utiliza el siguiente diagrama. Este modelo permite identificar las entidades principales del sistema: Ventas y Visitantes, junto con sus respectivos atributos o campos.

TB_VENTAS	
🔑	Id_venta
	Monto_total
	Metodo_pago
	Fecha_venta

TB_VISITANTES	
🔑	Id_visita
	Nom_visita
	Cant_bol
	hora_ent

ARQUITECTURA DEL SOFTWARE

El sistema Ticket Museum está desarrollado bajo la plataforma WPF (Windows Presentation Foundation) utilizando una arquitectura de capas lógica estructurada por desacoplamiento de archivos dentro de la solución. A diferencia de los entornos Windows Forms tradicionales, este proyecto separa la definición visual mediante lenguaje de marcado XAML de la lógica de control en código subyacente (Code-Behind en C#).

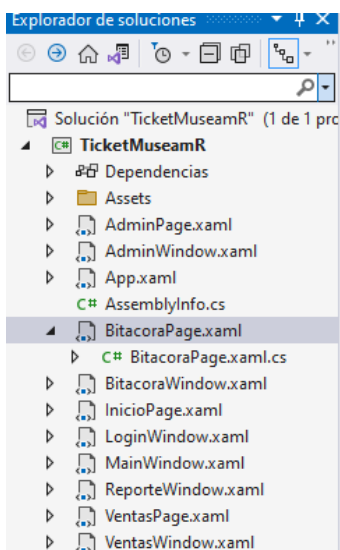
Organización de los componentes técnicos

1. Componentes de la Interfaz de Usuario (Presentación XAML):

- **Contenedores de Aplicación (Windows):** Archivos base que sirven exclusivamente como lienzos o marcos contenedores del ciclo de vida del software (MainWindow.xaml, LoginWindow.xaml, AdminWindow.xaml, BitacoraWindow.xaml, ReporteWindow.xaml y VentasWindow.xaml). Estos archivos no procesan lógica interna, actúan únicamente como los huéspedes visuales del sistema.
- **Páginas Modulares Activas (Pages):** Representan los módulos operativos reales donde se ejecuta la interacción. Estas incluyen AdminPage.xaml, BitacoraPage.xaml y VentasPage.xaml.

Lógica de Control Subyacente (Code-Behind):

- La interactividad del sistema y las reglas de negocio están programadas directamente en el código de respaldo en C# de las páginas modulares (por ejemplo, BitacoraPage.xaml.cs). Desde esta subcapa se capturan los eventos físicos del usuario (como el clic en "Actualizar Tabla" o "Cobrar") y se coordinan los flujos de información.



MÉTODOS PRINCIPALES DEL SISTEMA

Para la ejecución de las tareas operativas de la taquilla, el sistema implementa controladores de eventos en el código subyacente (Code-Behind) de las páginas modulares XAML. Estos métodos se encargan de capturar las entradas de la interfaz y gestionar las peticiones transaccionales hacia la base de datos BDMUSEO

Gestión de Autenticación de Usuarios:

- Ubicación: Módulo de acceso / Interfaz inicial.

Descripción Técnica: Al presionar el botón de ingreso, el método captura las credenciales introducidas en los campos de texto. Invoca los componentes de datos para abrir un canal seguro con SQL Server y ejecuta una consulta parametrizada. Si el servidor confirma la existencia del usuario y sus privilegios (cajero o administrador), el sistema redirige dinámicamente el marco de navegación hacia las páginas operativas principales (VentasPage o BitacoraPage), otorgando el acceso al sistema sin depender de ventanas externas.

- Registro de Transacciones de Boletaje (InsertarVenta):

Ubicación: VentasPage.xaml.cs.

Descripción Técnica: Controla la inserción de auditorías financieras en la tabla TB_VENTAS. Cuando el cajero confirma el cobro en la interfaz de ventas, el método extrae las variables del total calculado y el método de pago (Efectivo o Tarjeta) ingresados en los elementos visuales de la página, ejecutando una sentencia INSERT INTO TB_VENTAS utilizando parámetros estrictos para resguardar la integridad de los datos.

- Actualización de Historial en Tiempo Real (CargarBitacora):

Ubicación: BitacoraPage.xaml.cs.

Descripción Técnica: Este método automatiza el llenado del componente visual encargado de mostrar los reportes. Al activarse mediante el botón "Actualizar Tabla" visible en la interfaz, realiza una consulta directa SELECT * FROM TB_VISITANTES a la base de datos. Los registros recuperados se enlazan de forma directa a la propiedad ItemsSource del control DataGrid de WPF, refrescando la tabla de visitas inmediatamente en pantalla.

REQUERIMIENTOS DE INSTALACIÓN Y DESPLIEGUE

Para desplegar el sistema **Ticket Museum v1.0** en un entorno de producción (las taquillas del museo), se deben seguir estos pasos estructurados:

1. **Instalación del Gestor de Datos:** Instalar SQL Server Express (versión 2019 o superior) y ejecutar el script de creación de la base de datos BDMUSEO junto con las tablas TB_VENTAS y TB_VISITANTES.
2. **Configuración de la Cadena de Conexión:** Verificar que la instancia del servidor local de base de datos coincida con la configurada en los controladores de las páginas operativas de la solución.
3. **Compilación del Ejecutable:** Generar el archivo instalable o ejecutable compilando el proyecto en modo **Release** desde Visual Studio. El resultado generará una carpeta con el archivo TicketMuseumR.exe y sus dependencias, lista para copiarse en la computadora de la taquilla.

MANEJO DE EXCEPCIONES Y ERRORES (SEGURIDAD TÉCNICA)

El sistema implementa políticas de tolerancia a fallos mediante bloques de control try-catch en todos los métodos del Code-Behind que realizan peticiones de datos:

- **Excepciones de Conexión (SQLException):** Si el servidor SQL Server se encuentra apagado o inaccesible al presionar "Actualizar Tabla" en BitacoraPage.xaml, el sistema captura el error y despliega un mensaje controlado (MessageBox) informando al usuario, evitando que la aplicación sufra un cierre inesperado (crash).
- **Validación de Datos:** Antes de enviar las sentencias INSERT, el software valida que los campos de texto no apunten a valores nulos o vacíos (string.IsNullOrEmpty), previniendo errores de restricción NOT NULL desde el cliente.

MANTENIMIENTO FUTURO Y ESCALABILIDAD

El diseño modular basado en páginas XAML de WPF garantiza que el sistema pueda escalar eficientemente:

- **Mantenimiento Correctivo:** Cualquier cambio en las reglas de negocio o consultas SQL solo requiere modificar los archivos de respaldo específicos (como VentasPage.xaml.cs), sin alterar la interfaz visual global.
- **Escalabilidad:** La arquitectura permite agregar nuevos módulos de manera sencilla (por ejemplo, un InventarioPage.xaml o ConfiguracionPage.xaml) e indexarlos directamente en el sistema de navegación del contenedor principal MainWindow.xaml.